An Empirical Performance Benchmarking Evaluation of Locally Deployed Retrieval-Augmented Generation Constraints

Prajjwal Singh
Independent Technical Framework Study (Work in Progress)
Lucknow, India
prajjwals.acad@gmail.com

***Abstract*—This framework details a localized technical workbench designed to monitor the operational limits of privacy-preserving Retrieval-Augmented Generation (RAG) system pipelines on consumer hardware. Orchestrated via Python, LangChain, ChromaDB, and an offline Llama 3 instance running over local computing frames, the benchmark monitors the distinct processing impacts of token-based text segmentation boundaries. Our metrics isolate background resource overhead from inference execution loops. This document outlines an ongoing engineering study tracking structural system constraints rather than a generalized validated optimization conclusion.**

I. Experimental Methodology & System Boundary

To ensure reproducibility, our framework architecture decouples text ingestion, data index retrieval, and local synthesis text blocks:

- **Token Ingestion Routing:** Document text segments are extracted via PyPDF. To maintain literal linguistic parameter tracking, raw strings are divided using a token-aware context text splitter (`TokenTextSplitter`) calculated over the `all-MiniLM-L6-v2` vector model space.
- **Hardware Sandbox Specs:** All computation intervals were recorded locally on a consumer-grade laptop sandbox running macOS, Python, and localized Llama 3 model weights managed via Ollama daemon terminals.

II. Empirical Evaluation & Latency Segregation

Rather than combining pipeline performance measurements into an ambiguous sum, our benchmarking metrics track database initialization latencies completely separately from downstream model response extraction. The testing runs evaluate three distinct token window boundaries ($C \in \{200, 400, 600\}$ tokens), executing multiple continuous trials to log cache fluctuations.

TABLE I
Empirical Hardware Latency Metrics Across Token-Aware Vector Scales

Token Size (Chunks)	Retrieval Depth (Top-K Depth)	Database Indexing Latency (s)	Mean Inference Latency (s)
200	3	5.404	1.120
400	3	4.941	0.787
600	3	5.094	0.889

A. Parameter Scaling Analysis & Real Data Evaluation

Our directly recorded empirical measurements indicate that token-aware ingestion optimization stabilizes indexing parameters within a range of 4.94 to 5.40 seconds on local hardware frames. Configuring fine token boundaries ($C = 200$) yields an indexing latency baseline of 5.404 seconds. As chunk size limits scale up to $C = 400$, the latency requirements drop to a minimum of 4.941 seconds due to lower indexing node generation density.

Downstream generation inference latencies scale efficiently cross-sectionally, remaining bounded between a mean of 0.787 and 1.120 seconds. The unanswerable testing categories successfully verified prompt-grounding stability by consistently invoking fallback strings without hallucinating peripheral global weights. As an empirical workbench evaluation framework, these latency profiles trace specific local processing throughput configurations rather than absolute universal quantitative answer-quality metrics.

III. Current Scope & Ongoing Research Directions

As an independent study in progress, the framework is bounded by its narrow dataset evaluation pool and specific hardware environment parameters. Future iterations will focus on integrating automated similarity metrics to log answer-quality deviations across varying model architectures.